

LABORATORY

LAB 09

Distance Measurement with HC-SR04

Reading Distance with the Ultrasonic Device API in EduFramework

S32K144 · MaaZEDU

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	Ultrasonic Distance Measurement	2
2.2	TRIG and ECHO Signals	2
2.3	From ECHO Time to Distance	3
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	Pin Mapping	3
3.3	Connecting the HC-SR04	4
4	EduFramework API	4
4.1	<code>ultrasonicBegin()</code>	4
4.2	<code>ultrasonicRead()</code>	5
5	Lab Exercise	5
5.1	Requirements	5
5.2	Program	5
5.3	Verification	6
6	Extension	7
6.1	Reading Distance in Inches with <code>ultrasonicReadInch()</code>	7
6.2	Filtering Measurements with <code>ultrasonicReadFiltered()</code>	7
6.3	Additional Ultrasonic APIs	8
7	References	8

1 Introduction

1.1 Lab Overview

An ultrasonic sensor can determine the distance to an object without direct physical contact. The HC-SR04 performs a measurement by transmitting an ultrasonic signal, waiting for the reflected signal to return, and using the signal travel time to determine distance [1].

In EduFramework, the process of generating the trigger signal, measuring the response time, and converting the result into distance is encapsulated in the Ultrasonic Device API. The application can therefore work directly with a distance value in centimeters instead of handling each measurement step manually [6].

The lab exercise uses `ultrasonicBegin()` to initialize the HC-SR04 and `ultrasonicRead()` to read the distance, then displays the result on the Serial Monitor. Measurements in other units, multi-sample filtering, and timeout configuration are reserved for the Extension section.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the basic principle of ultrasonic distance measurement.
2. Explain the roles of the `TRIG` and `ECHO` signals on the HC-SR04.
3. Explain the relationship between the ultrasonic response time and the distance to an object.
4. Use `ultrasonicBegin()` and `ultrasonicRead()` to measure distance in centimeters.
5. Build and verify a distance monitoring application using the Serial Monitor.

2 Background

2.1 Ultrasonic Distance Measurement

The HC-SR04 uses ultrasonic waves to measure the distance to an object. When a measurement begins, the module transmits an ultrasonic signal at approximately `40 kHz`. The signal travels through the air, reflects from the object, and returns to the sensor. The elapsed time between transmission and reception of the reflection is used to determine distance [1] [2].

The measurement process can be represented as:

Sensor → Ultrasonic wave → Object → Reflected wave → Sensor

Because the signal must travel from the sensor to the object and then return, the measured time corresponds to a round trip rather than only the one-way distance from the sensor to the object [3].

2.2 TRIG and ECHO Signals

The HC-SR04 uses two main digital signals during a measurement. `TRIG` is the input used to start a measurement, while `ECHO` is the output that represents the response time of the ultrasonic signal. A `HIGH` pulse of at least approximately `10 µs` on `TRIG` causes the module to transmit a burst of eight `40 kHz` ultrasonic cycles [3] [2].

After transmission, the width of the `HIGH` pulse on `ECHO` varies according to the travel and reflection time of the ultrasonic wave. Therefore, measuring the interval during which `ECHO` remains `HIGH` provides the information needed to calculate distance [1] [3].

In the EduFramework Ultrasonic Device, `TRIG` is configured as a Digital Output and `ECHO` as a Digital Input. The Device API sequentially generates the trigger pulse, waits for `ECHO`, measures the pulse width, and processes the result [6].

2.3 From ECHO Time to Distance

Let t be the time required for the ultrasonic wave to travel from the sensor to the object and return, and let v be the speed of sound. The one-way distance d is determined by:

$$d = \frac{v \times t}{2}$$

The division by 2 is required because the measured time includes both the outgoing and return paths of the signal. For the HC-SR04, the `ECHO` time in microseconds can be converted to distance using the approximate relationship $\mu\text{s} / 58 = \text{cm}$ [3].

The current EduFramework implementation uses the factor 0.017 to convert the `ECHO` pulse width in microseconds into centimeters:

```
distance_cm = duration_us × 0.017
```

This value corresponds to the distance conversion based on the round-trip travel time of the ultrasonic wave [3] [6].

The Ultrasonic Device also uses a timeout to prevent the program from waiting indefinitely when a valid `ECHO` signal is not received. With the default configuration, the timeout is 30000 μs ; when a measurement is invalid, the distance API returns `ULTRASONIC_INVALID_DISTANCE_CM` [6].

3 Hardware Setup

3.1 Required Hardware

The lab exercise uses the MaaZEDU Development Board and an HC-SR04 module. The module has four pins: `VCC`, `GND`, `TRIG`, and `ECHO`. The `TRIG` and `ECHO` signals are connected to EduFramework GPIO Logical Pins [1] [6].

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
HC-SR04 ultrasonic sensor	Ultrasonic sensor module used to measure distance to an object.
Jumper wires	Connect power and the <code>TRIG</code> and <code>ECHO</code> signals between HC-SR04 and MaaZEDU.
USB Cable	Connect the board to the computer for power, programming, and Serial Monitor communication.

Table 9.3.1. Hardware used in the lab exercise

3.2 Pin Mapping

The lab uses `GPIO2` for the `TRIG` signal and `GPIO1` for the `ECHO` signal. On the MaaZEDU Development Board, `GPIO2` maps to `PTD14`, while `GPIO1` maps to `PTD17` [5].

Component	Logical Pin	MCU Pin	Function
HC-SR04 TRIG	GPI02	PTD14	Digital Output
HC-SR04 ECHO	GPI01	PTD17	Digital Input

Table 9.3.2. HC-SR04 signal mapping used in the lab exercise

3.3 Connecting the HC-SR04

In the hardware configuration verified for this lab exercise, the HC-SR04 is powered from 5V , shares GND with MaaZEDU, has TRIG connected to GPI02 , and has ECHO connected to GPI01 . The HC-SR04 uses a 5V supply in the operating configuration described for the module [1] [2] .

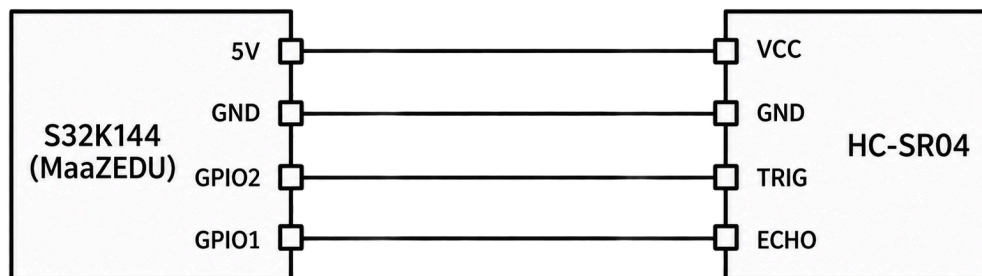


Figure 9.3.1. HC-SR04 connection to the MaaZEDU Development Board

Note

The lab exercise uses the GPI02 and GPI01 Logical Pins instead of manipulating raw microcontroller pin names directly. This keeps the program at the EduFramework API level and follows the MaaZEDU pin mapping [5] [6] .

4 EduFramework API

The main exercise requires only two Ultrasonic Device APIs: `ultrasonicBegin()` and `ultrasonicRead()` . Time measurements, unit conversion, multi-sample filtering, and timeout configuration are reserved for the Extension section [6] .

4.1 `ultrasonicBegin()`

`ultrasonicBegin()` initializes the default ultrasonic sensor using two Logical Pins for TRIG and ECHO . The API configures TRIG as an output, ECHO as an input, and uses the default timeout of the Ultrasonic Device [6] .

`ultrasonicBegin`

Initialize the default HC-SR04 sensor and configure the TRIG and ECHO pins.

Syntax

```
ultrasonicBegin(trigPin, echoPin);
```

Parameters

Parameter	Type / accepted value	Description
<code>trigPin</code>	Digital Logical Pin	Digital pin connected to the HC-SR04 TRIG pin.
<code>echoPin</code>	Digital Logical Pin	Digital pin connected to the HC-SR04 ECHO pin.

Return value

No return value.

Example:

```
ultrasonicBegin(GPI02, GPI01);
```

4.2 `ultrasonicRead()`

`ultrasonicRead()` performs one measurement using the default sensor and returns the distance in centimeters. If a valid ECHO pulse is not measured before the timeout, the API returns `ULTRASONIC_INVALID_DISTANCE_CM`, whose value is `-1.0F` [6].

`ultrasonicRead`

Perform a measurement with the HC-SR04 and return the distance in centimeters.

Syntax

```
ultrasonicRead();
```

Return value

Distance in centimeters when the measurement succeeds; `ULTRASONIC_INVALID_DISTANCE_CM` when a timeout or invalid measurement occurs.

Example:

```
float distance = ultrasonicRead();
```

5 Lab Exercise

5.1 Requirements

Build a program that measures distance with the HC-SR04 using TRIG connected to GPI02 and ECHO connected to GPI01. The program initializes `Serial1` at a baud rate of `9600`, reads the distance in centimeters, and updates the result on the Serial Monitor every `500 ms`.

If the measurement is invalid, the program displays `Distance: Timeout`. After the program operates correctly, change the distance between the sensor and a flat object to observe the corresponding change in the result.

5.2 Program

In `src/main.c`, implement the program that has been verified on hardware as follows:

```
#include "Arduino.h"
#include "ultrasonic.h"

int main(void)
{
    float distance = ULTRASONIC_INVALID_DISTANCE_CM;

    setup();

    Serial1_begin(9600U);
    ultrasonicBegin(GPI02, GPI01);

    Serial1_println("HC-SR04 Distance Monitor");

    while (1)
    {
        distance = ultrasonicRead();

        if (ULTRASONIC_INVALID_DISTANCE_CM != distance)
        {
            Serial1_print("Distance: ");
            Serial1_printFloat(distance);
            Serial1_println(" cm");
        }
        else
        {
            Serial1_println("Distance: Timeout");
        }

        delay(500U);
    }

    return 0;
}
```

Listing 9.5.1. Program for measuring distance with the HC-SR04

`setup()` initializes the underlying components of EduFramework. `Serial1_begin()` initializes the Serial Monitor channel, while `ultrasonicBegin(GPI02, GPI01)` initializes the sensor with `GPI02` assigned to `TRIG` and `GPI01` assigned to `ECHO` [6].

In the main loop, `ultrasonicRead()` triggers the sensor, measures the `ECHO` time, and converts the result into centimeters. The program checks the return value before printing the result so that a valid measurement can be distinguished from a timeout [6].

The application processing flow can be summarized as:

Object → HC-SR04 → ECHO time → `ultrasonicRead()` → Distance → `Serial1` → Serial Monitor [6].

5.3 Verification

Build and upload the program to the MaaZEDU Development Board, then open the Serial Monitor at a baud rate of `9600`. Place a flat object in front of the HC-SR04 and observe the distance value. Move the object farther from the sensor and then closer again to verify the trend in the measured result.

The Serial Monitor output has the following form:

```
HC-SR04 Distance Monitor
Distance: 10.24 cm
Distance: 10.19 cm
Distance: 20.36 cm
Distance: 30.41 cm
```

When a valid measurement is not received within the timeout period, the program displays:

```
Distance: Timeout
```

EXPECTED RESULT

The Serial Monitor updates the result approximately every `500 ms`. When the object is moved farther from the sensor, the distance value tends to increase; when the object is moved closer, the value tends to decrease. If a valid `ECHO` signal is not received before the timeout, the program displays `Timeout`. This behavior is consistent with the HC-SR04 response time measurement principle and the processing implemented in the Ultrasonic Device [1] [6].

6 Extension

The main exercise uses only distance measurements in centimeters. The Ultrasonic Device also provides APIs for unit conversion, observing response time, changing the timeout, and processing multiple measurement samples [6].

6.1 Reading Distance in Inches with `ultrasonicReadInch()`

`ultrasonicReadInch()` performs a measurement with the default sensor and returns the distance in inches. The API uses the distance measurement result and performs the unit conversion inside the Device [6].

`ultrasonicReadInch`

Read the distance from the HC-SR04 and return the result in inches.

Syntax

```
ultrasonicReadInch();
```

Return value

Distance in inches when the measurement succeeds; `ULTRASONIC_INVALID_DISTANCE_CM` when the measurement is invalid.

Example:

```
float distanceInch = ultrasonicReadInch();
```

Extension: Modify the program to display the distance in both centimeters and inches on the Serial Monitor. The result can be presented in the following form:

```
Distance: 25.40 cm
Distance: 10.00 inch
```

6.2 Filtering Measurements with `ultrasonicReadFiltered()`

A single measurement can vary between consecutive readings. `ultrasonicReadFiltered()` collects multiple distance samples before returning a processed result. With the current default configuration, the Device

collects 20 samples, sorts them, removes the 5 lowest and 5 highest samples, and then averages the remaining samples [6].

ultrasonicReadFiltered

Collect multiple distance samples and return a processed value from the middle group of samples.

Syntax

```
ultrasonicReadFiltered();
```

Return value

Filtered distance in centimeters when valid data is available; `ULTRASONIC_INVALID_DISTANCE_CM` if no valid result is obtained.

Example:

```
float filteredDistance = ultrasonicReadFiltered();
```

Extension: Keep the object at a fixed position and compare the results of `ultrasonicRead()` and `ultrasonicReadFiltered()` on the Serial Monitor.

Example:

```
Raw: 25.36 cm
Filtered: 25.18 cm
```

6.3 Additional Ultrasonic APIs

EduFramework also provides several additional APIs [6]:

- `ultrasonicReadDuration()` returns the ECHO pulse width in microseconds.
- `ultrasonicSetTimeout(timeoutUs)` changes the timeout of the default measurement.
- The `Ultrasonic_Begin()`, `Ultrasonic_ReadCm()`, `Ultrasonic_ReadInch()`, and `Ultrasonic_ReadCmFiltered()` API group uses an `Ultrasonic_t` object to support multiple sensor instances in the same application.

These APIs are not required for the main exercise. They are useful when the application needs to observe the response time directly, adjust the timeout, or use multiple ultrasonic sensors in one system [6].

Extension: Use `ultrasonicReadDuration()` to display both the ECHO duration and the distance, then observe how the two values change as the object is moved.

7 References

- [1] New York University, *Lab: Ultrasonic Distance Sensor*, ITP Physical Computing.
- [2] Oregon State University, *Sonar Rangefinder HC-SR04*, TekBots.
- [3] SparkFun Electronics, *HC-SR04 Ultrasonic Sensor Datasheet*, Technical Datasheet.
- [4] NXP Semiconductors, *S32K1xx Series Cookbook*, AN5413, Rev. 5, 2020.
- [5] MaaZEDU, *MaaZEDU Development Board Guide*.
- [6] EduFramework, *Ultrasonic Device API*, EduFramework Source Code.